

Introduction to Databases

Lecture 7:

Functional Dependencies, Boyce-Codd Normal Form

Floris Geerts

Today's Menu

- Theory behind relational database design
- Functional dependencies as a tool to express potential redundancies
- Normal forms express desirable properties
 - Boyce-Codd Normal Form
 - 3rd Normal Form
- Decomposing relations to achieve normal form
 - Lossless
 - Dependency preserving

Example: Relation with Redundancies

SID	sname	CID	cname	Semester	lecturer	grade
0001	Jan Janssens	IDB	Introduction to Databases	S1, 2014	Floris Geerts	8
0002	Els Van Elzen	IDB	Introduction to Databases	S1, 2014	Floris Geerts	16
0001	Jan Janssens	IDB	Introduction to Databases	S1, 2015	Jon Sneyers	12
0003	An Aerts	IDB	Introduction to Databases	S1, 2015	Jon Sneyers	13
0004	Yves Peters	IDB	Introduction to Databases	S1, 2015	Jon Sneyers	17
0005	Karen Daems	IP	Inleiding Programmeren	S1, 2015	Jon Sneyers	13
0005	Karen Daems	IDB	Introduction to Databases	S1, 2016	Toon Calders	7
0006	John Claes	IP	Inleiding Programmeren	S1, 2016	Toon Calders	15
0007	Ivo De Wolf	IP	Inleiding Programmeren	S1, 2018	Toon Calders	16
...	...	...	...	...	...	...

- Which properties of this table make it unsuitable for storing our data?

What Is Wrong With This Relation?

- Redundancy: names of lecturers, courses, students recorded multiple times
- Potential inconsistencies: two tuples with same SID but different sname
- Update anomalies: changing the name of a course requires multiple tuples to change
- Deletion anomalies: naively deleting students from the database may cause information about courses to get lost

SID	sname	CID	cname	Semester	lecturer	grade
0001	Jan Janssens	IDB	Introduction to Databases	S1, 2014	Floris Geerts	8
0002	Els Van Elzen	IDB	Introduction to Databases	S1, 2014	Floris Geerts	16
0001	Jan Janssens	IDB	Introduction to Databases	S1, 2015	Jon Sneyers	12
...	...	...	...	...	...	...

A Better Way to Store this Data

This particular form of decomposition is called **Boyce-Codd Normal Form (BCNF)**.

SID	sname
0001	Jan Janssens
0002	Els Van Elzen
0003	An Aerts
0004	Yves Peters
0005	Karen Daems
0006	John Claes
0007	Ivo De Wolf
...	...

CID	Semester	lecturer
IDB	S1, 2014	Floris Geerts
IDB	S1, 2015	Jon Sneyers
IP	S1, 2015	Jon Sneyers
IDB	S1, 2016	Toon Calders
IP	S1, 2016	Toon Calders
...	...	...

CID	cname
IDB	Introduction to Databases
IP	Inleiding Programmeren
...	...

SID	CID	Semester	grade
0001	IDB	S1, 2014	8
0002	IDB	S1, 2014	16
0001	IDB	S1, 2015	12
0003	IDB	S1, 2015	13
0004	IDB	S1, 2015	17
0005	IP	S1, 2015	13
0005	IDB	S1, 2016	7
0006	IP	S1, 2016	15
0007	IP	S1, 2018	16
...	...	...	...

But WHY is B better than A?

- This lecture will equip you with theoretical notions that allow you to articulate in a precise way what is wrong with solution A
 - Functional dependencies, keys
- Using these notions we will be able to express the conditions a healthy relation has to satisfy
 - Boyce Codd Normal Form
- We will see techniques that allow us to decompose a relation into this normal form
 - While still holding the same information → lossless
 - Is always possible

Why is B better than A?

- In solution A, it holds that two tuples that have the same SID, need to agree on the sname.
- At the same time, however, it is possible to have two different tuples with the same SID
- **Hence, unless we write an assertion to check that whenever two tuples agree on SID, they also agree on sname, we are in peril!**

SID	sname	CID	cname	Semester	lecturer	grade
0001	Jan Janssens	IDB	Introduction to Databases	S1, 2014	Floris Geerts	8
0002	Els Van Elzen	IDB	Introduction to Databases	S1, 2014	Floris Geerts	16
0001	Jan Janssens	IDB	Introduction to Databases	S1, 2015	Jon Sneyers	12
...	...	...	...	...	...	...

Why is B better than A?

- In solution B, it also holds that two tuples that have the same SID, need to agree on the sname.
- However, it is **not** possible to have two different tuples with the same SID, because SID is a key in all relations containing sname

<u>SID</u>	sname
0001	Jan Janssens
0002	Els Van Elzen
...	...

<u>CID</u>	<u>Semester</u>	lecturer
IDB	S1, 2014	Floris Geerts
IDB	S1, 2015	Jon Sneyers
...	...	...

<u>CID</u>	cname
IDB	Introduction to Databases
IP	Inleiding Programmeren
...	...

<u>SID</u>	<u>CID</u>	<u>Semester</u>	grade
0001	IDB	S1, 2014	8
0002	IDB	S1, 2014	16
...	...	...	...

Functional Dependency

- In solution A, it holds that two tuples that have the same SID, need to agree on the sname.
- This is a so-called Functional Dependency (FD)
 - We express this as: $SID \rightarrow sname$

Other FDs: $CID \rightarrow cname$; $CID, Semester \rightarrow lecturer$;
 $SID, CID, semester \rightarrow grade$

SID	sname	CID	cname	Semester	lecturer	grade
0001	Jan Janssens	IDB	Introduction to Databases	S1, 2014	Floris Geerts	8
0002	Els Van Elzen	IDB	Introduction to Databases	S1, 2014	Floris Geerts	16
0001	Jan Janssens	IDB	Introduction to Databases	S1, 2015	Jon Sneyers	12
...	...	...	...	...	...	...

Functional Dependency

A **functional dependency (FD)** on a relation with schema (set of attributes) R is a statement of the form: $X \rightarrow Y$, with $X, Y \subseteq R$

A relation instance r **satisfies an FD $X \rightarrow Y$** iff: for all t_1, t_2 in r such that $t_1(X) = t_2(X)$, it holds that $t_1(Y) = t_2(Y)$ *

* $t_1(X) = t_2(X)$ is shorthand for: for all attributes $A \in X$, it holds that $t_1(A) = t_2(A)$

Example: Functional Dependencies

- Which of the following functional dependencies holds *in general* in our solution A?

$SID \rightarrow sname$ $CID \rightarrow cname$; $CID, Semester \rightarrow lecturer$;

$SID, CID, semester \rightarrow grade$

$CID \rightarrow lecturer$ $grade \rightarrow SID, CID$ $SID, sname \rightarrow SID$

$CID, sname, semester \rightarrow lecturer$. $SID, CID \rightarrow grade, sname$

SID	sname	CID	cname	Semester	lecturer	grade
0001	Jan Janssens	IDB	Introduction to Databases	S1, 2014	Floris Geerts	8
0002	Els Van Elzen	IDB	Introduction to Databases	S1, 2014	Floris Geerts	16
0001	Jan Janssens	IDB	Introduction to Databases	S1, 2015	Jon Sneyers	12
...	...	...	...	...	...	...

Example: Functional Dependencies

- Which of the following functional dependencies holds *in general* in our solution A?

**SID $\rightarrow$ sname CID $\rightarrow$ cname ; CID, Semester $\rightarrow$ lecturer ;
SID, CID, semester $\rightarrow$ grade**

CID $\rightarrow$ lecturer grade $\rightarrow$ SID, CID SID, sname $\rightarrow$ SID

CID, sname, semester $\rightarrow$ lecturer SID, CID $\rightarrow$ grade, sname

SID	sname	CID	cname	Semester	lecturer	grade
0001	Jan Janssens	IDB	Introduction to Databases	S1, 2014	Floris Geerts	8
0002	Els Van Elzen	IDB	Introduction to Databases	S1, 2014	Floris Geerts	16
0001	Jan Janssens	IDB	Introduction to Databases	S1, 2015	Jon Sneyers	12
...	...	...	...	...	...	...

Not all FDs are Equally Informative

Trivial vs. non-trivial FD's:

Let X and Y be sets of attributes of a relation r .

An FD $X \rightarrow Y$ is called

- **trivial**: if Y is a subset of X ; E.g., $AB \rightarrow B$
- **non-trivial**: if $Y - X$ is not empty; E.g., $AB \rightarrow BC$
- **completely non-trivial**: if non-trivial, X and Y are disjoint, e.g., $AB \rightarrow CD$

Implication of FDs

- Let F be a set of FDs. We say that F implies FD $X \rightarrow Y$ if every relation that satisfies all FDs in F , also satisfies $X \rightarrow Y$

For instance:

$\{ A \rightarrow B, B \rightarrow C \}$ implies $A \rightarrow C$

$\{ A \rightarrow BC \}$ implies $A \rightarrow B$ and $A \rightarrow C$

$\{ AB \rightarrow C \}$ does not imply $A \rightarrow C$

Implication of FDs

Proposition: $F = \{ A \rightarrow B, B \rightarrow C \}$ implies $A \rightarrow C$

Proof:

Let r be a relation that satisfies F , and let t_1 and t_2 be such that $t_1(A) = t_2(A)$.

Then $t_1(B) = t_2(B)$ // r satisfies $A \rightarrow B$

Then $t_1(C) = t_2(C)$ // r satisfies $B \rightarrow C$

Therefore, r also satisfies $A \rightarrow C$, because for any pair of tuples t_1, t_2 such that $t_1(A) = t_2(A)$, also $t_1(C) = t_2(C)$ holds.

QED

Implication of FDs

- $F = \{ AB \rightarrow C \}$ does not imply $A \rightarrow C$

Proof by counterexample:

We construct a relation r that satisfies F , but not $A \rightarrow C$:

A	B	C
1	1	1
1	2	2

Inference Rules

- An **inference rule** for FDs is a description showing how we can construct other FDs that are **implied** by a given set of FDs.

For instance:

- **Splitting rule**: $X \rightarrow Y \cup Z$ implies $X \rightarrow Y$ and $X \rightarrow Z$
- **Combining rule**: $X \rightarrow Y$ and $X \rightarrow Z$ implies $X \rightarrow Y \cup Z$
- A set of inference rules that allows to make all valid derivations is called an **axiomatization**

Armstrong's Axiomatization

X , Y , and Z denote sets of attributes

A1 Reflexivity: if $Y \subseteq X$, then $X \rightarrow Y$ holds

A2 Augmentation: if $X \rightarrow Y$ holds, then also $X \cup Z \rightarrow Y \cup Z$

A3 Transitivity: if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

These are called **Armstrong's Axioms**; they can be shown to be sound and complete for the derivation of functional dependencies

All valid derivations can be made by applying a sequence of these rules

Axioms: Splitting Rule

Splitting rule: $X \rightarrow Y \cup Z$ implies $X \rightarrow Y$ and $X \rightarrow Z$

A1 Reflexivity: if $Y \subseteq X$, then $X \rightarrow Y$ holds

A2 Augmentation: if $X \rightarrow Y$ holds, then also $X \cup Z \rightarrow Y \cup Z$

A3 Transitivity: if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

$$X \rightarrow YZ$$

Axioms: Splitting Rule

A1 Reflexivity: if $Y \subseteq X$, then $X \rightarrow Y$ holds

A2 Augmentation: if $X \rightarrow Y$ holds, then also $X \cup Z \rightarrow Y \cup Z$

A3 Transitivity: if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

Splitting rule: $X \rightarrow Y \cup Z$ implies $X \rightarrow Y$ and $X \rightarrow Z$

$$X \rightarrow YZ$$

$$\frac{\quad}{YZ \rightarrow Y} \text{ A1}$$

Axioms: Splitting Rule

A1 Reflexivity: if $Y \subseteq X$, then $X \rightarrow Y$ holds

A2 Augmentation: if $X \rightarrow Y$ holds, then also $X \cup Z \rightarrow Y \cup Z$

A3 Transitivity: if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

Splitting rule: $X \rightarrow Y \cup Z$ implies $X \rightarrow Y$ and $X \rightarrow Z$

$$\frac{X \rightarrow YZ \quad \frac{\quad}{YZ \rightarrow Y} \text{A1}}{\quad \text{A3}} X \rightarrow Y$$

Same for $X \rightarrow Z$

Axioms: Combining Rule

Combining rule: $X \rightarrow Y$ and $X \rightarrow Z$ implies $X \rightarrow Y \cup Z$

A1 Reflexivity: if $Y \subseteq X$, then $X \rightarrow Y$ holds

A2 Augmentation: if $X \rightarrow Y$ holds, then also $X \cup Z \rightarrow Y \cup Z$

A3 Transitivity: if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

$$X \rightarrow Y$$

$$X \rightarrow Z$$

Axioms: Combining Rule

Combining rule: $X \rightarrow Y$ and $X \rightarrow Z$ implies $X \rightarrow Y \cup Z$

A1 Reflexivity: if $Y \subseteq X$, then $X \rightarrow Y$ holds

A2 Augmentation: if $X \rightarrow Y$ holds, then also $X \cup Z \rightarrow Y \cup Z$

A3 Transitivity: if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

$$\frac{X \rightarrow Y}{X \rightarrow XY} \text{ A2}$$

$$X \rightarrow Z$$

Axioms: Combining Rule

Combining rule: $X \rightarrow Y$ and $X \rightarrow Z$ implies $X \rightarrow Y \cup Z$

A1 Reflexivity: if $Y \subseteq X$, then $X \rightarrow Y$ holds

A2 Augmentation: if $X \rightarrow Y$ holds, then also $X \cup Z \rightarrow Y \cup Z$

A3 Transitivity: if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

$$\frac{X \rightarrow Y}{X \rightarrow XY} \text{ A2}$$

$$\frac{X \rightarrow Z}{XY \rightarrow YZ} \text{ A2}$$

Axioms: Combining Rule

Combining rule: $X \rightarrow Y$ and $X \rightarrow Z$ implies $X \rightarrow Y \cup Z$

A1 Reflexivity: if $Y \subseteq X$, then $X \rightarrow Y$ holds

A2 Augmentation: if $X \rightarrow Y$ holds, then also $X \cup Z \rightarrow Y \cup Z$

A3 Transitivity: if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

$$\frac{\frac{X \rightarrow Y}{X \rightarrow XY} \text{ A2} \quad \frac{X \rightarrow Z}{XY \rightarrow YZ} \text{ A2}}{X \rightarrow YZ} \text{ A3}$$

Closure w.r.t. a Set of FDs

- Using Armstrong's axioms can be tedious; therefore:
- Let X be a set of attributes and F a set of FDs; the **closure of X with respect to F** , denoted $X^{+(F)}$ is defined as:
$$X^{+(F)} = \{ A \mid X \rightarrow A \text{ is implied by } F \}$$
- $X \rightarrow X^{+(F)}$ always holds

$X \rightarrow Y$ is implied by F if and only if $Y \subseteq X^{+(F)}$

Closure w.r.t. Set of FDs

- Let $F = \{ A \rightarrow C, B \rightarrow D, AD \rightarrow E, BE \rightarrow G, CH \rightarrow E \}$

$$AB^{+(F)} = \{ A, B, C, D, E, G \}$$

- F implies $AB \rightarrow EG$?
- Yes because EG is a subset of $AB^{+(F)}$

Easy Way to Compute Closure

Initialize $X^{+(F)}$ as X ;

Repeat until no new attributes are found:

Search for an FD of the form $U \rightarrow V$ in F

where U is a subset of $X^{+(F)}$ and V is not a subset of $X^{+(F)}$;

$X^{+(F)} := X^{+(F)} \cup V$;

Example: Computing Closure

Let $X=\{A,B\}$, $F=\{AB \rightarrow C, BC \rightarrow A, BC \rightarrow D, D \rightarrow E, CH \rightarrow B\}$.

- $X^{+(F)} = \{A,B\}$ (apply $AB \rightarrow C$)
- $X^{+(F)} = \{A,B,C\}$ (apply $BC \rightarrow D$)
- $X^{+(F)} = \{A,B,C,D\}$ (apply $D \rightarrow E$)
- $X^{+(F)} = \{A,B,C,D,E\}$

Example: Computing Closure

Let $X = \{B, D\}$,

$F = \{AB \rightarrow C, C \rightarrow A, BC \rightarrow D, ACD \rightarrow B, D \rightarrow EG, BE \rightarrow C, CG \rightarrow BD, CE \rightarrow AG\}$

- $X^{+(F)} = \{B, D, E, G\}$ (apply $D \rightarrow EG$)
- $X^{+(F)} = \{B, C, D, E, G\}$ (apply $BE \rightarrow C$)
- $X^{+(F)} = \{A, B, C, D, E, G\}$ (apply $C \rightarrow A$)

Defining Keys via FDs

- A set of attributes X is a **superkey** in a relation r with attributes R iff:
 $X \rightarrow R$ holds in r
(Notice that $X \rightarrow R$ is equivalent to $X \rightarrow (R \setminus X)$)
- X is a **candidate key** if X is a superkey and for no $X' \subset X$ is a superkey.
That is: X is a minimal set with the superkey property
- Defining a **primary key** in a relation schema is hence the same as stating a functional dependency from the primary key to all other attributes.
- **Q:** Characterize superkeys using closure of X relative to set F of FDs:
- X is a superkey in a relation with attributes R iff $X^{+(F)} = R$

Finding all Candidate Keys

- Any attribute that does not appear on the RHS of any FD should be part of any key
- Any key can be generated in the following way
 - Start from $X = \{ A \mid A \text{ appears in no RHS} \}$
 - Keep adding attributes not in X^+ to X , until $X^+ = R$
- Systematically go over all possibilities
 - To avoid generating duplicates, only add attributes to X that lexicographically come after all attributes already added to X
E.g., never add B to AC, as it will already be generated by adding C to AB
- Remove non-minimal sets afterwards

Example: Finding all Candidate Keys

$$F = \{ AB \rightarrow C, CD \rightarrow E, A \rightarrow E, DE \rightarrow A \}$$

B and D do not appear on RHS of any FD $\Rightarrow$ they must be in any key

$$BD^+ = BD$$

- **$ABD^+ = ABCDE$**
- **$BCD^+ = ABCDE$**
- **$BDE^+ = ABCDE$**

There are three candidate keys: ABD, BCD, and BDE

Example 2: Finding all Candidate Keys

$F = \{ C \rightarrow T, C \rightarrow R, HR \rightarrow C, HT \rightarrow R, CS \rightarrow G, HS \rightarrow R \}$

H and S do not appear in any RHS

$HS^+ = HSRCTG \rightarrow HS$ is the only key

Example 3: Finding all Candidate Keys

$F = \{ A \rightarrow B, BC \rightarrow E, D \rightarrow E, AE \rightarrow D, CDE \rightarrow A, BDE \rightarrow C \}$

All attributes appear on RHS

- $A^+ = AB$
 - $AC^+ = ABCDE$
 - $AD^+ = ABCDE$
 - $AE^+ = ABCDE$
- $B^+ = B$
 - $BC^+ = BCE$
 - $BCD^+ = ABCDE$
 - $BD^+ = ABCDE$
 - $BE^+ = BE$

Example 3: Finding all Candidate Keys

$F = \{ A \rightarrow B, BC \rightarrow E, D \rightarrow E, AE \rightarrow D, CDE \rightarrow A, BDE \rightarrow C \}$

- $C^+ = C$
 - $CD^+ = ABCDE$
 - $CE^+ = CE$
- $D^+ = DE$
- $E^+ = E$

Keys are: AC, AD, AE, BD, CD

Summary Functional Dependencies

- FD's are a form of **relation-level constraints**.
- FD's are a **property of a relation schema**, not that of a relation instance. They are assumed to be given along with the schema definition of that relation.
- FD's can help us to **identify keys** of a relation schema R.
 - X is a key of R, if the closure of attributes X wrt. the FD's covers all the attributes of R.

But WHY is B better than A?

- This lecture will equip you with theoretical notions that allow you to articulate in a precise way what is wrong with relation A

*Solution A has a relation in which a non-trivial functional dependency $X \rightarrow Y$ holds and X is not a superkey; for instance: **SID $\rightarrow$ sname***

SID	sname	CID	cname	Semester	lecturer	grade
0001	Jan Janssens	IDB	Introduction to Databases	S1, 2014	Floris Geerts	8
0002	Els Van Elzen	IDB	Introduction to Databases	S1, 2014	Floris Geerts	16
0001	Jan Janssens	IDB	Introduction to Databases	S1, 2015	Jon Sneyers	12
...	...	...	...	...	...	...

But WHY is B better than A?

- Using these notions we will be able to express the conditions a healthy relation has to satisfy

Assume a set of functional dependencies F is given. Our normal forms will all be with respect to a given set of constraints

A relation is in Boyce-Codd Normal Form (BCNF), if for every non-trivial FD $X \rightarrow Y$ in F it holds that X is a superkey of R .

A database is in BCNF if every relation in it is.

Decomposition

- We will see techniques that allow us to decompose a relation into this normal form
- A decomposition of a relation schema R is a set of subsets of that schema $R_1, \dots, R_k$, such that $R = R_1 \cup \dots \cup R_k$
- The decomposition is lossless w.r.t. a set of functional dependencies F if for every relation r with schema R satisfying the given FDs, it holds that $r = \pi_{R_1} r \bowtie \dots \bowtie \pi_{R_k} r$

The Chase: Testing if a Decomposition is Lossless

We want to test if $r = \pi_{R_1} r \bowtie \dots \bowtie \pi_{R_k} r$ holds if r satisfies F

Direction $r \subseteq \pi_{R_1} r \bowtie \dots \bowtie \pi_{R_k} r$ is trivially satisfied.

Indeed, take tuple t in r and define $t_i = \pi_{R_i}(t)$ for $i=1,2,\dots,k$. Then, we have that $t \in \pi_{R_1} r \bowtie \dots \bowtie \pi_{R_k} r$ because $t = t_1 \bowtie \dots \bowtie t_k$.

The Chase: Testing if a Decomposition is Lossless

We want to test if $r = \pi_{R_1} r \bowtie \dots \bowtie \pi_{R_k} r$ holds if r satisfies F

Direction $r \subseteq \pi_{R_1} r \bowtie \dots \bowtie \pi_{R_k} r$ is trivially satisfied.

We need to show that $\pi_{R_1} r \bowtie \dots \bowtie \pi_{R_k} r \subseteq r$

Let $t \in \pi_{R_1} r \bowtie \dots \bowtie \pi_{R_k} r$

Then there are tuples $t_i \in \pi_{R_i} r$ such that $t_i(A) = t(A)$ for all $A \in R_i$

Hence there must be tuples $t'_i \in r$ such that $t_i(A) = t(A)$ for all $A \in R_i$

The chase is a technique to check if the existence of such tuples t'_i implies t is in r as well, using F

Example 1: The Chase

We will illustrate with some examples:

- Decompose $R(A,B,C,D)$ under $FD = \{ A \rightarrow CD, B \rightarrow D, BC \rightarrow A \}$ into $R_1 = AB$ and $R_2 = ACD$

To get tuple (a,b,c,d) in $\pi_{AB}r \bowtie \pi_{ACD}r$, we need to have (a,b) in $\pi_{AB}r$ and (a,c,d) in $\pi_{ACD}r$.

Hence, we need tuples (a,b,c_1,d_1) and (a,b_1,c,d) in r :

A	B	C	D
a	b	c ₁	d ₁
a	b ₁	c	d

Example 1: The Chase

- We will illustrate with some examples:
- Decompose $R(A,B,C,D)$ under $FD = \{ A \rightarrow C, B \rightarrow D, BC \rightarrow A \}$ into $R_1 = AB$ and $R_2 = ACD$

To get tuple (a,b,c,d) in $\pi_{AB}r \bowtie \pi_{ACD}r$, we need to have (a,b) in $\pi_{AB}r$ and (a,c,d) in $\pi_{ACD}r$.

Hence, we need tuples (a,b,c_1,d_1) and (a,b_1,c,d) in r :

A	B	C	D
a	b	c_1	d_1
a	b_1	c	d

Example 1: The Chase

- We will illustrate with some examples:
- Decompose $R(A,B,C,D)$ under $FD = \{ A \rightarrow C, B \rightarrow D, BC \rightarrow A \}$ into $R_1 = AB$ and $R_2 = ACD$

To get tuple (a,b,c,d) in $\pi_{AB}r \bowtie \pi_{ACD}r$, we need to have (a,b) in $\pi_{AB}r$ and (a,c,d) in $\pi_{ACD}r$.

Hence, we need tuples (a,b,c_1,d_1) and (a,b_1,c,d) in r :

A	B	C	D
a	b	c	d
a	b ₁	c	d

Example 1: The Chase

- We will illustrate with some examples:
- Decompose $R(A,B,C,D)$ under $FD = \{ A \rightarrow C, B \rightarrow D, BC \rightarrow A \}$ into $R_1 = AB$ and $R_2 = ACD$

To get tuple (a,b,c,d) in $\pi_{AB}r \bowtie \pi_{ACD}r$, we need to have (a,b) in $\pi_{AB}r$ and (a,c,d) in $\pi_{ACD}r$.

Hence, we need tuples (a,b,c_1,d_1) and (a,b_1,c,d) in r :

A	B	C	D
a	b	c	d
a	b ₁	c	d

(a,b,c,d) must also be in r !

Example 2: The Chase

We will illustrate with some examples:

- Decompose $R(A,B,C,D)$ into $R_1(A,D)$, $R_2(A,C)$ and $R_3(B,C,D)$
- $F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow A\}$

A	B	C	D
a	b_1	c_1	d
a	b_2	c	d_1
a_1	b	c	d

Example 2: The Chase

We will illustrate with some examples:

- Decompose $R(A,B,C,D)$ into $R_1(A,D)$, $R_2(A,C)$ and $R_3(B,C,D)$
- $F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow A\}$

A	B	C	D
a	b_1	c_1	d
a	b_1	c	d_1
a_1	b	c	d

Example 2: The Chase

We will illustrate with some examples:

- Decompose $R(A,B,C,D)$ into $R_1(A,D)$, $R_2(A,C)$ and $R_3(B,C,D)$
- $F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow A\}$

A	B	C	D
a	b_1	c	d
a	b_1	c	d_1
a_1	b	c	d

Example 2: The Chase

We will illustrate with some examples:

- Decompose $R(A,B,C,D)$ into $R_1(A,D)$, $R_2(A,C)$ and $R_3(B,C,D)$
- $F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow A\}$

A	B	C	D
a	b_1	c	d
a	b_1	c	d_1
a	b	c	d

Example 2: The Chase

We will illustrate with some examples:

- Decompose $R(A,B,C,D)$ into $R_1(A,D)$, $R_2(A,C)$ and $R_3(B,C,D)$
- $F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow A\}$

A	B	C	D
a	b_1	c	d
a	b_1	c	d_1
a	b	c	d

(a,b,c,d) must also be in r!

Example 3: The Chase

We will illustrate with some examples:

- Decompose $R(A,B,C,D)$ into $R_1(A,D)$, $R_2(A,C)$, $R_3(C,D)$, and $R_4(B,C)$
- $F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow A\}$

A	B	C	D
a	b_1	c_1	d
a	b_2	c	d_1
a_1	b_3	c	d
a_2	b	c	d_2

Example 3: The Chase

We will illustrate with some examples:

- Decompose $R(A,B,C,D)$ into $R_1(A,D)$, $R_2(A,C)$, $R_3(C,D)$, and $R_4(B,C)$
- $F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow A\}$

A	B	C	D
a	b_1	c_1	d
a	b_1	c	d_1
a_1	b_3	c	d
a_2	b	c	d_2

Example 3: The Chase

We will illustrate with some examples:

- Decompose $R(A,B,C,D)$ into $R_1(A,D)$, $R_2(A,C)$, $R_3(C,D)$, and $R_4(B,C)$
- $F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow A\}$

A	B	C	D
a	b_1	c	d
a	b_1	c	d_1
a_1	b_3	c	d
a_2	b	c	d_2

Example 3: The Chase

We will illustrate with some examples:

- Decompose $R(A,B,C,D)$ into $R_1(A,D)$, $R_2(A,C)$, $R_3(C,D)$, and $R_4(B,C)$
- $F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow A\}$

A	B	C	D
a	b_1	c	d
a	b_1	c	d_1
a	b_3	c	d
a_2	b	c	d_2

Example 3: The Chase

We will illustrate with some examples:

- Decompose $R(A,B,C,D)$ into $R_1(A,D)$, $R_2(A,C)$, $R_3(C,D)$, and $R_4(B,C)$
- $F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow A\}$

A	B	C	D
a	b_1	c	d
a	b_1	c	d_1
a	b_1	c	d
a_2	b	c	d_2

Example 3: The Chase

We will illustrate with some examples:

- Decompose $R(A,B,C,D)$ into $R_1(A,D)$, $R_2(A,C)$, $R_3(C,D)$, and $R_4(B,C)$
- $F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow A\}$

A	B	C	D
a	b_1	c	d
a	b_1	c	d_1
a	b_1	c	d
a_2	b	c	d_2

Example 3: The Chase

We will illustrate with some examples:

- Decompose $R(A,B,C,D)$ into $R_1(A,D)$, $R_2(A,C)$, $R_3(C,D)$, and $R_4(B,C)$
- $F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow A\}$

A	B	C	D
a	b_1	c	d
a	b_1	c	d_1
a	b_1	c	d
a_2	b	c	d_2

Example 3: The Chase

We will illustrate with some examples:

- Decompose $R(A,B,C,D)$ into $R_1(A,D)$, $R_2(A,C)$, $R_3(C,D)$, and $R_4(B,C)$
- $F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow A\}$

A	B	C	D
a	b_1	c	d
a	b_1	c	d_1
a	b_1	c	d
a_2	b	c	d_2

- **No more changes are possible**

Example 3: The Chase

We will illustrate with some examples:

- Decompose $R(A,B,C,D)$ into $R_1(A,D)$, $R_2(A,C)$, $R_3(C,D)$, and $R_4(B,C)$
- $F = \{ A \rightarrow B, B \rightarrow C, CD \rightarrow A \}$

- **No more changes are possible**

- Hence, relation satisfies F
- Does not contain (a,b,c,d)
- But the decomposition does!

A	B	C	D
a	b_1	c	d
a	b_1	c	d_1
a	b_1	c	d
a_2	b	c	d_2

- **We have a counter example for the losslessness!**

Desirable properties of decompositions

- **Lossless:** decomposing $R(A,B,C,D)$ under $FD = \{ A \rightarrow CD, B \rightarrow D, BC \rightarrow A \}$ into $R_1(A,B)$ and $R_2(A,C,D)$ is lossless because for every legal instance R it holds that $R_1 \bowtie R_2 = R$

A	B	C	D
1	1	5	6
1	2	5	6

 =

A	B
1	1
1	2

 $\bowtie$

A	C	D
1	5	6

Algorithm for Computing a Decomposition in BCNF

The following algorithm decomposes any relation scheme R into BCNF w.r.t. a given set of FDs F :

Given: a relation schema R , and a set of FDs F

If there is a non-trivial FD $X \rightarrow Y$ in F^+ ,
where X is not a superkey of R :

- Decompose R into:
 - $S(X^{+(F)})$
 - $T(R \setminus (X^{+(F)} \setminus X))$
- Recursively decompose S and T into BCNF.

Essential to check for all dependencies in F^+ , not just F !

Else: stop

$R(A,B,C,D,E,F)$

Not in BCNF

$FDs = \{A \rightarrow DE, B \rightarrow D, C \rightarrow E, DE \rightarrow F\}$

Select non-trivial FD: e.g, $B \rightarrow D$

- Compute $B^+ = \{B, D\}$, $B^+ \setminus B = \{D\}$, $\{A, B, C, D, E, F\} - D = \{A, B, C, E, F\}$
- Decompose in $R_1(B, D)$ and $R_2(A, B, C, E, F)$: R_1 in BCNF, R_2 not yet

Select non-trivial FD (for R_2), e.g., $C \rightarrow E$.

- Compute $C^+ = \{C, E\}$, $C^+ \setminus C = \{E\}$, $\{A, B, C, E, F\} - E = \{A, B, C, F\}$
- Decompose R_2 in $R_{21}(C, E)$ and $R_{22}(A, B, C, F)$ R_{21} in BCNF, R_{22} not yet. Why?

We need to consider F^+ and $A \rightarrow F$ is in F^+ .

- Compute $A^+ = \{A, D, E, F\}$, $A^+ \setminus A = \{D, E, F\}$, $\{A, B, C, F\} - \{D, E, F\} = \{A, B, C\}$
- Decompose R_2 in $R_{221}(A, F)$ and $R_{222}(A, B, C)$

Now all parts are in BCNF

Example: Decomposition into BCNF

- $R(\text{SID}, \text{sname}, \text{CID}, \text{cname}, \text{lect}, \text{sem}, \text{grade})$

$$F = \{ \quad \text{SID} \rightarrow \text{sname}, \quad \text{CID} \rightarrow \text{cname}, \quad \text{CID}, \text{sem} \rightarrow \text{lect}, \\ \text{SID}, \text{CID}, \text{sem} \rightarrow \text{grade} \quad \}$$

Pick rule $\text{SID} \rightarrow \text{sname}$

$R_1(\text{SID}, \text{sname})$

$R_2(\text{SID}, \text{CID}, \text{cname}, \text{lect}, \text{sem}, \text{grade})$

Example: Decomposition into BCNF

- $R(\text{SID}, \text{sname}, \text{CID}, \text{cname}, \text{lect}, \text{sem}, \text{grade})$

$F = \{ \text{SID} \rightarrow \text{sname}, \quad \text{CID} \rightarrow \text{cname}, \quad \text{CID}, \text{sem} \rightarrow \text{lect},$
 $\text{SID}, \text{CID}, \text{sem} \rightarrow \text{grade} \}$

Pick rule $\text{SID} \rightarrow \text{sname}$

$R_1(\text{SID}, \text{sname})$

$R_2(\text{SID}, \text{CID}, \text{cname}, \text{lect}, \text{sem}, \text{grade})$

Pick rule $\text{CID}, \text{sem} \rightarrow \text{lect}$ $(\text{CID}, \text{sem})^+ = (\text{CID}, \text{sem}, \text{lect}, \text{cname})$

$R_{21}(\text{CID}, \text{sem}, \text{lect}, \text{cname})$

$R_{22}(\text{SID}, \text{CID}, \text{sem}, \text{grade})$

Example: Decomposition into BCNF

- $R(\text{SID}, \text{sname}, \text{CID}, \text{cname}, \text{lect}, \text{sem}, \text{grade})$

$F = \{ \text{SID} \rightarrow \text{sname}, \text{CID} \rightarrow \text{cname}, \text{CID}, \text{sem} \rightarrow \text{lect}, \text{SID}, \text{CID}, \text{sem} \rightarrow \text{grade} \}$

Pick rule $\text{SID} \rightarrow \text{sname}$

$R_1(\text{SID}, \text{sname})$

$R_2(\text{SID}, \text{CID}, \text{cname}, \text{lect}, \text{sem}, \text{grade})$

Pick rule $\text{CID}, \text{sem} \rightarrow \text{lect}$

$R_{21}(\text{CID}, \text{sem}, \text{lect}, \text{cname})$

Pick rule $\text{CID} \rightarrow \text{cname}$ $\text{CID}^+ = (\text{CID}, \text{cname})$

$R_{211}(\text{CID}, \text{sem}, \text{lect})$

$R_{212}(\text{CID}, \text{cname})$

$R_{22}(\text{SID}, \text{CID}, \text{sem}, \text{grade})$

Example: Decomposition into BCNF

- $R(\text{SID}, \text{sname}, \text{CID}, \text{cname}, \text{lect}, \text{sem}, \text{grade})$

$F = \{ \text{SID} \rightarrow \text{sname}, \text{CID} \rightarrow \text{cname}, \text{CID}, \text{sem} \rightarrow \text{lect}, \text{SID}, \text{CID}, \text{sem} \rightarrow \text{grade} \}$

Pick rule $\text{SID} \rightarrow \text{sname}$

$R_1(\text{SID}, \text{sname})$

$R_2(\text{SID}, \text{CID}, \text{cname}, \text{lect}, \text{sem}, \text{grade})$

Pick rule $\text{CID}, \text{sem} \rightarrow \text{lect}$

$R_{21}(\text{CID}, \text{sem}, \text{lect}, \text{cname})$

Pick rule $\text{CID} \rightarrow \text{cname}$

$R_{211}(\text{CID}, \text{sem}, \text{lect})$

$R_{212}(\text{CID}, \text{cname})$

$R_{22}(\text{SID}, \text{CID}, \text{sem}, \text{grade})$

Conclusion

- Functional dependencies as a tool to express redundancies at schema level
 - Closures as a convenient way to determine implication, and keys
- FDs can flag problems in our relation:
 - Non-trivial $X \rightarrow Y$ with X not a superkey
- Boyce-Codd Normal Form indicates a well-designed relation not having this problem
- Good news: whenever a relation does not satisfy BCNF we can solve this by decomposing
 - We can ensure decomposition is lossless
 - Bonus: Chase as a technique to check if a decomposition is lossless
 - And decomposition is in BCNF